

EE446-TinyML Assignment 1

Author: Sparsh Dadhich

Github: https://github.com/sdadhich04/EE-446-TinyML-/tree/main/Assignment1_EE446

Problem 1: DNN and Wine Classification (80 points)

(e) For the final compression strategy, I combined knowledge distillation with full int8 quantization since both techniques compress the model in different ways. Knowledge distillation reduced the architecture size by training a smaller student network with only 1,027 parameters instead of the original 3,011, while int8 quantization reduced the storage size of each parameter by converting values from float32 to int8. The student model was trained using both the hard one-hot labels and the teacher model's soft probability distributions, which gave the smaller network more information about class relationships and confidence levels than normal training would. After training the student model, I applied full integer quantization using `tf.lite.Optimize.DEFAULT`, a representative dataset generated from `X_train_scaled`, `TFLITE_BUILTINS_INT8`, and forced both the input and output tensor types to `tf.int8`. During inference, the input data had to be manually quantized using the tensor scale and zero point before being passed into the interpreter, and the output tensor had to be dequantized back into float32 values before selecting the predicted class with `np.argmax`. The final KD + int8 model achieved a TFLite size of only 3.68 KB, which was the smallest model out of every compression method tested and significantly smaller than the standalone int8 model at 5.76 KB. Even with this aggressive compression, the model still achieved about 98% accuracy with only a single misclassification in the confusion matrix, showing that combining knowledge distillation with quantization preserved almost all predictive performance while drastically reducing model size.

Problem 2: Exploring Edge Impulse (20 points)

1. Edge Impulse's free tier supports several types of learning blocks for both supervised and unsupervised machine learning workflows. Under supervised learning, it supports classification, regression, and object detection models, where classification predicts discrete categories, regression predicts continuous numerical values, and object detection identifies both the class and location of objects inside an image. Example models include Keras classification networks, regression networks, MobileNetV2, and FOMO object detection. The platform also supports unsupervised learning methods such as clustering and anomaly detection through techniques like autoencoders and K-means clustering. In addition, transfer learning is heavily used through pretrained image models, allowing smaller datasets to still achieve strong performance by fine-tuning existing networks instead of training from scratch.

2. The classifier section in Edge Impulse supports several neural network layer types commonly used in TensorFlow and Keras models. Dense layers are fully connected layers used for learning

high-level patterns and generating classification outputs, while convolutional layers are mainly used for image and signal processing by detecting spatial features such as edges or textures. Depthwise convolution layers provide lightweight convolutions optimized for TinyML and embedded systems with lower computation cost. Pooling layers reduce feature-map size to decrease computation and overfitting, flatten layers convert multidimensional tensors into 1D vectors before classification, and dropout layers randomly disable neurons during training to improve generalization. Batch normalization layers stabilize training by normalizing activations, activation layers such as ReLU and softmax introduce nonlinearity, and recurrent layers like LSTMs can process sequential or time-series sensor data.

3. The deployment section in Edge Impulse provides several model compression and optimization methods designed for embedded and TinyML systems. Quantization methods such as int8, float16, and dynamic range quantization reduce model size by lowering numerical precision of weights and activations, which significantly reduces memory usage and inference cost. Pruning removes low-importance weights to create sparse models with fewer active parameters, while the EON compiler further optimizes inference graphs to reduce RAM usage, flash size, and latency on microcontrollers. Edge Impulse also supports TensorFlow Lite conversion and hardware-specific optimizations for platforms such as ARM CMSIS-NN, TensorRT, Edge TPU, and Arduino deployments. Together, these optimizations allow machine learning models to run efficiently on highly resource-constrained embedded devices.

4. Edge Impulse supports synthetic data generation and augmentation for multiple input modalities including image, audio, and sensor/time-series data. For image data, augmentation methods include flipping, rotation, cropping, brightness adjustment, contrast modification, and noise injection to improve robustness against environmental variation. Audio augmentation methods include background noise addition, pitch shifting, time stretching, and time shifting, while time-series sensor data can be augmented using scaling, jittering, Gaussian noise injection, signal shifting, and time warping. Synthetic data is important because collecting large real-world embedded datasets is often difficult, expensive, and time-consuming, especially for edge devices. By artificially increasing dataset diversity, synthetic data helps reduce overfitting, improve generalization, and make machine learning models more robust to real-world operating conditions and sensor noise.